UNLOX ACADEMY

VLSI Design Program, September 2026 Batch

MINI PROJECT REPORT

VERILOG MINI PROJECT ASSIGNMENT

Project 1: 1:4 Demultiplexer Design

Project 2: 3:8 Decoder Design

Project 3: 4-bit ALU Design

Subject: Digital Design using Verilog HDL

Modelled in Verilog HDL with testbenches and simulation waveforms

Student Name	Thipperudra K T
Roll No.	4SM23EE037
Date	6 November 2026
Instructor	Kuldeep Sompura

1. Project 1: 1:4 Demultiplexer Design

1.1 Aim

To design a 1:4 demultiplexer using Verilog HDL, implement it using both the dataflow and the behavioral model, verify all possible input combinations with a testbench, and capture the output waveforms.

1.2 Theory

A demultiplexer does the opposite job of a multiplexer: it takes a single data input and routes it to one of several outputs. The output is chosen by the select lines. A 1:4 demultiplexer has one data input *d*, two select lines *s[1:0]* and four outputs *y[3:0]*. The selected output follows the data input while the other three outputs stay at logic 0.

Boolean expressions of the outputs:

```
y0 = d . s1' . s0'      y1 = d . s1' . s0
y2 = d . s1  . s0'      y3 = d . s1  . s0
```

1.3 Block Diagram



Fig. 1: Block diagram of the 1:4 demultiplexer

1.4 Truth Table

Table 1: Truth table of the 1:4 demultiplexer

d	s1	s0	y3	y2	y1	y0
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

When d = 0 all outputs are 0. When d = 1, exactly one output goes high, and the select value s1s0 decides which one (00 selects y0, 01 selects y1, 10 selects y2 and 11 selects y3).

1.5 Verilog Design Code

Dataflow model. The outputs are written as continuous assignments using the Boolean expressions above. The design has no always block and no registers.

```
`timescale 1ns / 1ps
// 1:4 Demultiplexer - Dataflow model
module demux_1to4_dataflow(
    input        d,
    input  [1:0] s,
    output [3:0] y
);
    assign y[0] = d & ~s[1] & ~s[0];
    assign y[1] = d & ~s[1] &  s[0];
    assign y[2] = d &  s[1] & ~s[0];
    assign y[3] = d &  s[1] &  s[0];
endmodule
```

Fig. 2: Dataflow model of the 1:4 demultiplexer

Behavioral model. An *always @(*)* block first clears all outputs and then a *case* statement on *s* passes the data input *d* to the selected output. Clearing the outputs at the start avoids unintended latches.

```
`timescale 1ns / 1ps
// 1:4 Demultiplexer - Behavioral model
module demux_1to4_behavioral(
    input            d,
    input      [1:0] s,
    output reg [3:0] y
);
    always @(*) begin
        y = 4'b0000;
        case (s)
            2'b00: y[0] = d;
            2'b01: y[1] = d;
            2'b10: y[2] = d;
            2'b11: y[3] = d;
        endcase
    end
endmodule
```

Fig. 3: Behavioral model of the 1:4 demultiplexer

1.6 Testbench

The testbench instantiates both models with the same inputs. A for loop applies all 8 combinations of {d, s} (loop value 0 to 7) with a delay of 10 ns each, so that every input combination is covered and the two outputs can be compared.

```
`timescale 1ns / 1ps
module tb_demux_1to4;
    reg        d;
    reg  [1:0] s;
    wire [3:0] y_df, y_bh;
    integer    i;

    demux_1to4_dataflow   DUT1 (.d(d), .s(s), .y(y_df));
    demux_1to4_behavioral DUT2 (.d(d), .s(s), .y(y_bh));

    initial begin
        $dumpfile("demux.vcd");
        $dumpvars(0, tb_demux_1to4);
        $monitor("t=%0t  d=%b s=%b  y_dataflow=%b  y_behavioral=%b",
                 $time, d, s, y_df, y_bh);
        for (i = 0; i < 8; i = i + 1) begin
            {d, s} = i[2:0];      // all 8 input combinations
            #10;
        end
        $finish;
    end
endmodule
```

Fig. 4: Testbench of the 1:4 demultiplexer

1.7 Simulation Output

```
t=0  d=0 s=00  y_dataflow=0000  y_behavioral=0000
t=10  d=0 s=01  y_dataflow=0000  y_behavioral=0000
t=20  d=0 s=10  y_dataflow=0000  y_behavioral=0000
t=30  d=0 s=11  y_dataflow=0000  y_behavioral=0000
t=40  d=1 s=00  y_dataflow=0001  y_behavioral=0001
t=50  d=1 s=01  y_dataflow=0010  y_behavioral=0010
t=60  d=1 s=10  y_dataflow=0100  y_behavioral=0100
t=70  d=1 s=11  y_dataflow=1000  y_behavioral=1000
```

Fig. 5: Simulation output of the 1:4 demultiplexer

The dataflow and the behavioral model give identical outputs for all 8 input combinations.

1.8 Output Waveform

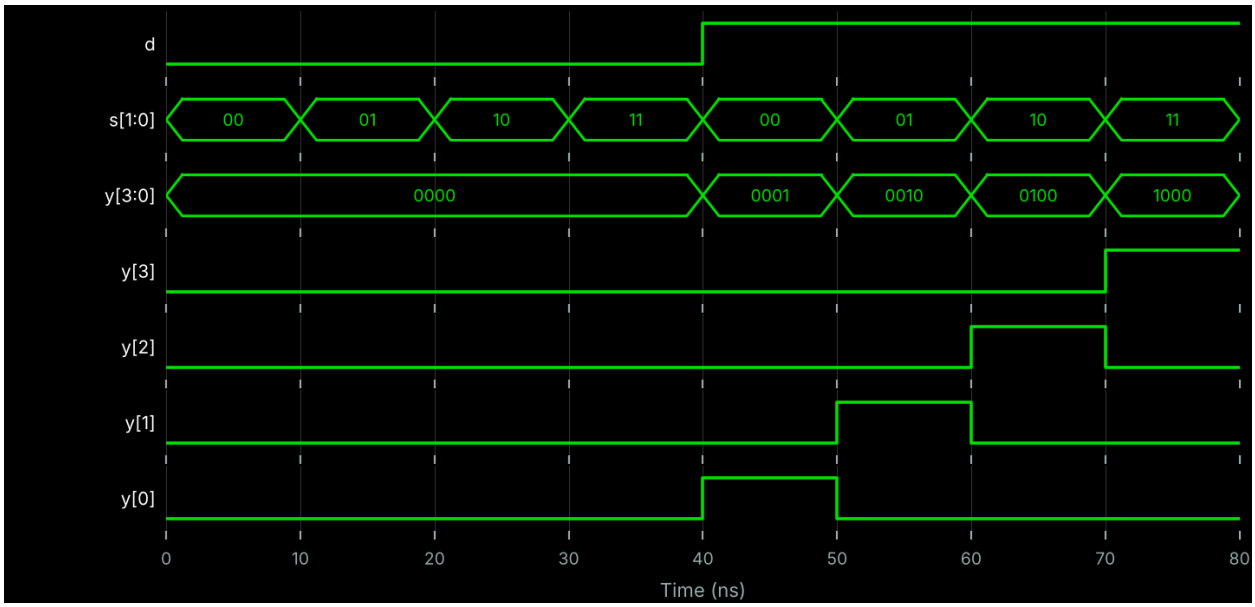


Fig. 6: Output waveform of the 1:4 demultiplexer

In the waveform, one bit of y[3:0] follows d at a time, and the selected bit changes with s[1:0] at every 10 ns step.

1.9 Result

The 1:4 demultiplexer was designed in the dataflow and the behavioral model, and the simulation confirmed the truth table for all input combinations.

2. Project 2: 3:8 Decoder Design

2.1 Aim

To design a 3:8 decoder with an enable input using behavioral modeling in Verilog HDL, verify all input combinations and enable conditions with a testbench, and capture the output waveforms.

2.2 Theory

A decoder converts an n-bit binary input into 2^n output lines, of which exactly one is active for each input value. The 3:8 decoder has three input lines *a[2:0]* and eight output lines *y[7:0]*. The enable input *en* controls the decoder: when en = 0 all outputs are 0 whatever the input is, and when en = 1 the output line whose number equals the binary input goes high (active-high outputs).

2.3 Block Diagram



Fig. 7: Block diagram of the 3:8 decoder with enable

2.4 Truth Table

Table 2: Truth table of the 3:8 decoder with enable

en	a2	a1	a0	y7	y6	y5	y4	y3	y2	y1	y0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

2.5 Verilog Design Code

The design uses an *always @(*)* block. The enable is checked first; if it is low the output is cleared, otherwise a *case* statement on *a* sets the matching output bit. The default branch keeps the output defined for any unexpected input.

```
`timescale 1ns / 1ps
// 3:8 Decoder with enable - Behavioral model
module decoder_3to8(
    input            en,
    input      [2:0] a,
    output reg [7:0] y
);
    always @(*) begin
        if (!en)
            y = 8'b00000000;
        else begin
            case (a)
                3'b000: y = 8'b00000001;
                3'b001: y = 8'b00000010;
                3'b010: y = 8'b00000100;
                3'b011: y = 8'b00001000;
                3'b100: y = 8'b00010000;
                3'b101: y = 8'b00100000;
                3'b110: y = 8'b01000000;
                3'b111: y = 8'b10000000;
                default: y = 8'b00000000;
            endcase
        end
    end
endmodule
```

Fig. 8: Behavioral model of the 3:8 decoder

2.6 Testbench

The testbench applies all 16 combinations of {en, a}. The first 8 values (0 to 80 ns) have en = 0 and check the disabled condition; the last 8 values (80 to 160 ns) have en = 1 and check every decoded output.

```
`timescale 1ns / 1ps
module tb_decoder_3to8;
    reg        en;
    reg  [2:0] a;
    wire [7:0] y;
    integer    i;

    decoder_3to8 DUT (.en(en), .a(a), .y(y));

    initial begin
        $dumpfile("decoder.vcd");
        $dumpvars(0, tb_decoder_3to8);
        $monitor("t=%0t  en=%b a=%b  y=%b", $time, en, a, y);
        for (i = 0; i < 16; i = i + 1) begin
            {en, a} = i[3:0];     // en=0 for first 8, en=1 for last 8
            #10;
        end
        $finish;
    end
endmodule
```

Fig. 9: Testbench of the 3:8 decoder

2.7 Simulation Output

```
t=0  en=0 a=000  y=00000000
t=10  en=0 a=001  y=00000000
t=20  en=0 a=010  y=00000000
t=30  en=0 a=011  y=00000000
t=40  en=0 a=100  y=00000000
t=50  en=0 a=101  y=00000000
t=60  en=0 a=110  y=00000000
t=70  en=0 a=111  y=00000000
t=80  en=1 a=000  y=00000001
t=90  en=1 a=001  y=00000010
t=100  en=1 a=010  y=00000100
t=110  en=1 a=011  y=00001000
t=120  en=1 a=100  y=00010000
t=130  en=1 a=101  y=00100000
t=140  en=1 a=110  y=01000000
t=150  en=1 a=111  y=10000000
```

Fig. 10: Simulation output of the 3:8 decoder

2.8 Output Waveform

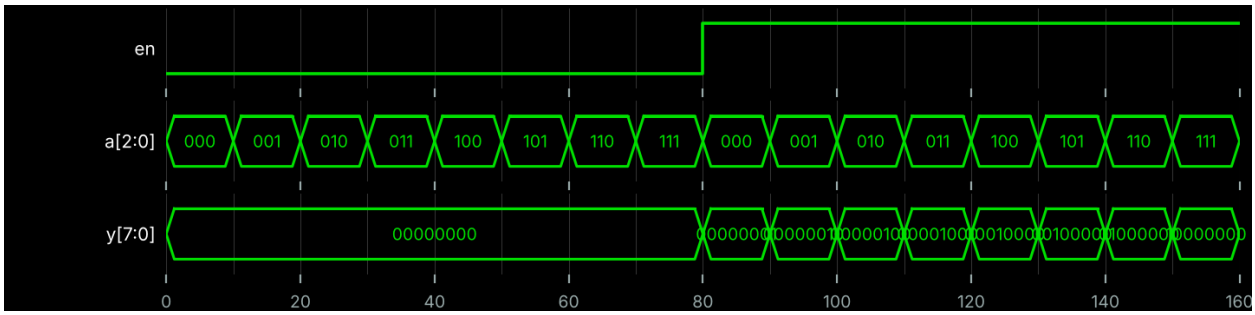


Fig. 11: Output waveform of the 3:8 decoder (en = 0 for 0 to 80 ns, en = 1 for 80 to 160 ns)

The output bus stays at 00000000 while en = 0. After en becomes 1 a single bit moves from y[0] to y[7] as the input a counts from 000 to 111.

2.9 Result

The 3:8 decoder with enable was designed in behavioral style, and the simulation confirmed the truth table for all input combinations and both enable conditions.

3. Project 3: 4-bit ALU Design

3.1 Aim

To design a 4-bit Arithmetic Logic Unit (ALU) in Verilog HDL that performs addition, subtraction, bitwise AND, OR, XOR, NOT, left shift and right shift, selecting the operation with a case statement, verify all operations with a testbench, and capture the output waveforms.

3.2 Theory

An ALU is the part of a processor that performs arithmetic and logic operations. This ALU has two 4-bit operands *a* and *b*, a 3-bit operation select *op*, a 4-bit result *y* and a carry/borrow output *cout*. The select code chooses one of eight operations. For addition, *cout* is the carry out of bit 3; for subtraction, *cout* = 1 means that a borrow occurred (a < b). NOT, left shift and right shift work on operand *a* only. The shifts move the bits by one place and fill the vacated bit with 0.

3.3 Block Diagram

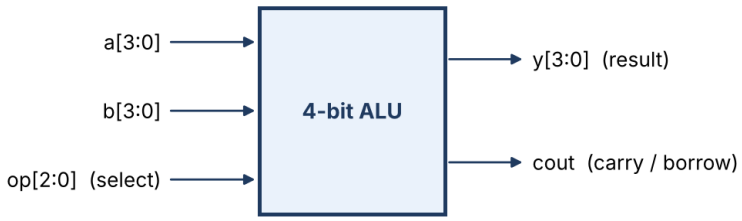


Fig. 12: Block diagram of the 4-bit ALU

3.4 Truth Table

Table 3: Operation (function) table of the 4-bit ALU

op[2:0]	Operation	Result
000	Addition	y = a + b
001	Subtraction	y = a - b
010	Bitwise AND	y = a & b
011	Bitwise OR	y = a | b
100	Bitwise XOR	y = a ^ b
101	Bitwise NOT	y = ~a
110	Left shift	y = a << 1
111	Right shift	y = a >> 1

Table 4: Result table for the test vectors used in the testbench

Operation	op	a	b	y	cout
ADD	000	1010 (10)	0011 (3)	1101 (13)	0
SUB	001	1010 (10)	0011 (3)	0111 (7)	0
AND	010	1010 (10)	0011 (3)	0010 (2)	0
OR	011	1010 (10)	0011 (3)	1011 (11)	0
XOR	100	1010 (10)	0011 (3)	1001 (9)	0
NOT	101	1010 (10)	0011 (3)	0101 (5)	0
LEFT SHIFT	110	1010 (10)	0011 (3)	0100 (4)	0
RIGHT SHIFT	111	1010 (10)	0011 (3)	0101 (5)	0
ADD (carry)	000	1111 (15)	0001 (1)	0000 (0)	1
SUB (borrow)	001	0011 (3)	1010 (10)	1001 (9)	1

For example, 1010 + 0011 = 1101 (10 + 3 = 13); 1111 + 0001 gives 0000 with cout = 1 (carry); and 0011 - 1010 gives 1001 with cout = 1 (borrow).

3.5 Verilog Design Code

A single *always @(*)* block contains a *case* statement on *op*. For addition and subtraction the result is assigned to the concatenation {cout, y}, so that the fifth bit is captured as carry or borrow. In subtraction both operands are extended by one bit before subtracting.

```
`timescale 1ns / 1ps
// 4-bit ALU - operation selected using a case statement
module alu_4bit(
    input      [3:0] a,
    input      [3:0] b,
    input      [2:0] op,
    output reg [3:0] y,
    output reg       cout      // carry (add) / borrow (subtract)
);
    always @(*) begin
        cout = 1'b0;
        case (op)
            3'b000: {cout, y} = a + b;                    // Addition
            3'b001: {cout, y} = {1'b0, a} - {1'b0, b};    // Subtraction
            3'b010: y = a & b;                            // Bitwise AND
            3'b011: y = a | b;                            // Bitwise OR
            3'b100: y = a ^ b;                            // Bitwise XOR
            3'b101: y = ~a;                               // Bitwise NOT
            3'b110: y = a << 1;                           // Left shift
            3'b111: y = a >> 1;                           // Right shift
            default: y = 4'b0000;
        endcase
    end
endmodule
```

Fig. 13: Verilog code of the 4-bit ALU

3.6 Testbench

The testbench first applies a = 1010 and b = 0011 with each of the 8 select codes, 10 ns apart. Two more vectors then test the carry case (15 + 1) and the borrow case (3 - 10).

```
`timescale 1ns / 1ps
module tb_alu_4bit;
    reg  [3:0] a, b;
    reg  [2:0] op;
    wire [3:0] y;
    wire       cout;

    alu_4bit DUT (.a(a), .b(b), .op(op), .y(y), .cout(cout));

    initial begin
        $dumpfile("alu.vcd");
        $dumpvars(0, tb_alu_4bit);
        $monitor("t=%0t  op=%b a=%b b=%b  y=%b cout=%b",
                 $time, op, a, b, y, cout);
        a = 4'b1010; b = 4'b0011;          // a = 10, b = 3
        op = 3'b000; #10;                  // ADD
        op = 3'b001; #10;                  // SUB
        op = 3'b010; #10;                  // AND
        op = 3'b011; #10;                  // OR
        op = 3'b100; #10;                  // XOR
        op = 3'b101; #10;                  // NOT
        op = 3'b110; #10;                  // LEFT SHIFT
        op = 3'b111; #10;                  // RIGHT SHIFT
        a = 4'b1111; b = 4'b0001; op = 3'b000; #10;   // ADD with carry
        a = 4'b0011; b = 4'b1010; op = 3'b001; #10;   // SUB with borrow
        $finish;
    end
endmodule
```

Fig. 14: Testbench of the 4-bit ALU

3.7 Simulation Output

```
t=0  op=000 a=1010 b=0011  y=1101 cout=0
t=10  op=001 a=1010 b=0011  y=0111 cout=0
t=20  op=010 a=1010 b=0011  y=0010 cout=0
t=30  op=011 a=1010 b=0011  y=1011 cout=0
t=40  op=100 a=1010 b=0011  y=1001 cout=0
t=50  op=101 a=1010 b=0011  y=0101 cout=0
t=60  op=110 a=1010 b=0011  y=0100 cout=0
t=70  op=111 a=1010 b=0011  y=0101 cout=0
t=80  op=000 a=1111 b=0001  y=0000 cout=1
t=90  op=001 a=0011 b=1010  y=1001 cout=1
```

Fig. 15: Simulation output of the 4-bit ALU

3.8 Output Waveform

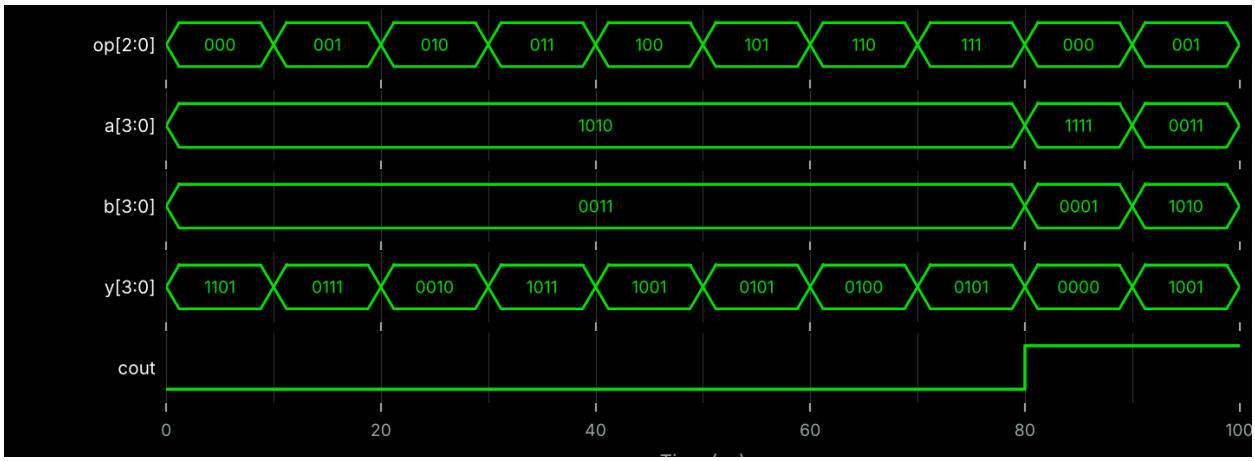


Fig. 16: Output waveform of the 4-bit ALU

The result y[3:0] changes at every 10 ns step as op[2:0] changes, and cout goes high only for the carry and borrow test vectors.

3.9 Result

The 4-bit ALU performed all eight operations correctly, and the simulation matched the expected results in Table 4.